Lab15-1 AI Assisted Coding

2303A51804

B-28

Task1:

Use AI to build a RESTful API for managing student records.

Prompt:

generate a python code to build a RESTful API for managing student records

The API should support the following operations:

GET/students - Retrieve a list of all students

POST/students - Create a new student record

PUT/students/{id} - Update an existing student record

DELETE/students/{id} - Delete a student record

use an in-memory data structure (list or dictionary) to store records and ensure API responses are in JSON format. Include error handling for cases such as trying to update or delete a non-existent student record.

CODE:

lab_15_1.py > ...

```python
#generate a python code to build a RESTful API for managing student records
# The API should support the following operations:
#GET/students - Retrieve a list of all students
#POST/students - Create a new student record
#PUT/students/{id} - Update an existing student record
#DELETE/students/{id} - Delete a student record
#use an in-memory data structure (list or dictionary) to store records and ensure API responses are in JSON format. Include error h
from flask import Flask, jsonify, request
app = Flask(__name__)
students = {}
@app.route('/')
def home():
    return jsonify({"message": "Welcome to the Student Records API!"})
@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(list(students.values()))
@app.route('/students', methods=['POST'])
def create_student():
    data = request.get_json()
    student_id = data.get('id')
    if student_id in students:
        return jsonify({'error': 'Student already exists'}), 400
    students[student_id] = data
    return jsonify(data), 201
@app.route('/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    if student_id not in students:
        return jsonify({'error': 'Student not found'}), 404
```

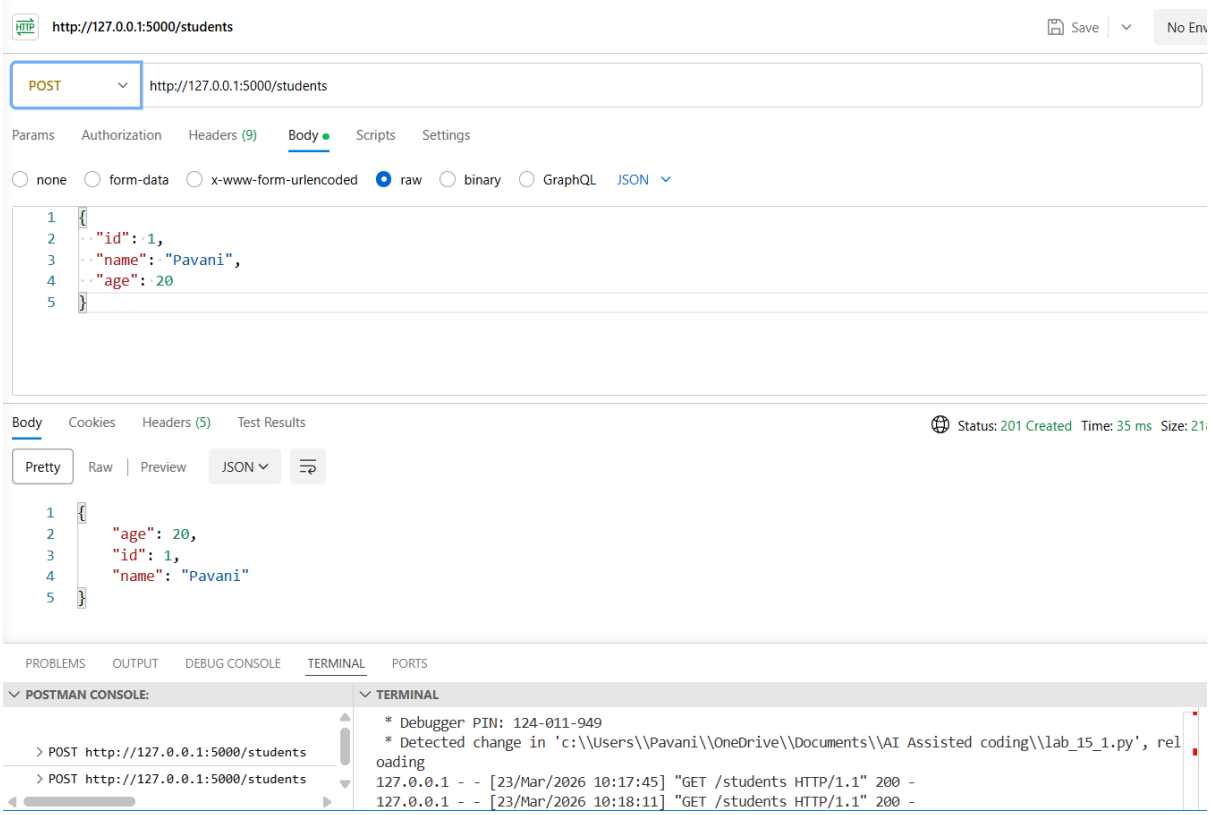
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```python
def get_students():
    return jsonify(list(students.values()))
@app.route('/students', methods=['POST'])
def create_student():
    data = request.get_json()
    student_id = data.get('id')
    if student_id in students:
        return jsonify({'error': 'Student already exists'}), 400
    students[student_id] = data
    return jsonify(data), 201
@app.route('/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    if student_id not in students:
        return jsonify({'error': 'Student not found'}), 404
    data = request.get_json()
    students[student_id] = data
    return jsonify(data)
@app.route('/students/<int:student_id>', methods=['DELETE'])
def delete_student(student_id):
    if student_id not in students:
        return jsonify({'error': 'Student not found'}), 404
    del students[student_id]
    return jsonify({'message': 'Student deleted'})
if __name__ == '__main__':
    app.run(debug=True)

```

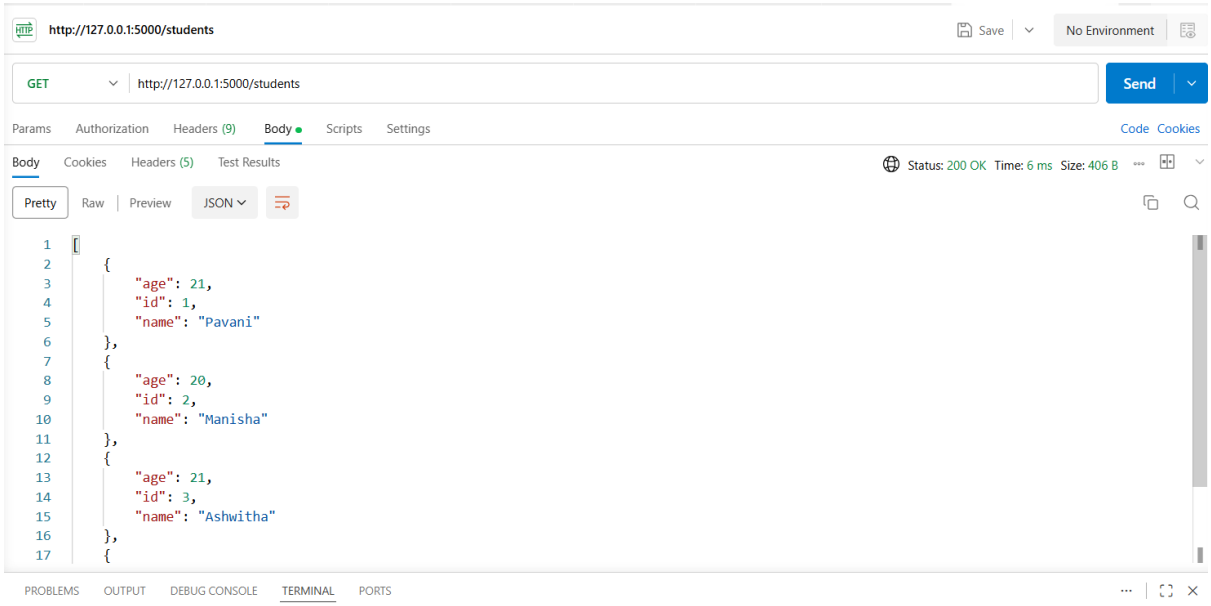
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

OUTPUT:

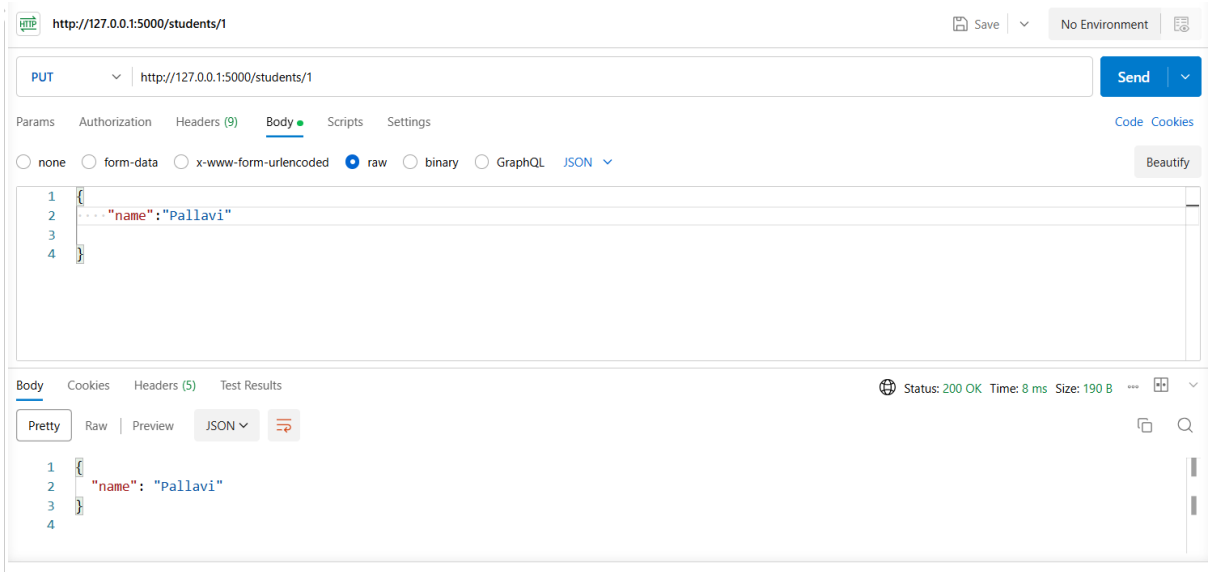
Post:


http://127.0.0.1:5000/students
Save
No Env
POST
http://127.0.0.1:5000/students
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"id": 1,
"name": "Pavani",
"age": 20
}
Body
Cookies
Headers (5)
Test Results
Status: 201 Created
Time: 35 ms
Size: 21
Pretty
Raw
Preview
JSON
{
"age": 20,
"id": 1,
"name": "Pavani"
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE:
POST http://127.0.0.1:5000/students
POST http://127.0.0.1:5000/students
TERMINAL
* Debugger PIN: 124-011-949
* Detected change in 'c:\\Users\\Pavani\\OneDrive\\Documents\\AI Assisted coding\\lab_15_1.py', reloading
127.0.0.1 - - [23/Mar/2026 10:17:45] "GET /students HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 10:18:11] "GET /students HTTP/1.1" 200 -

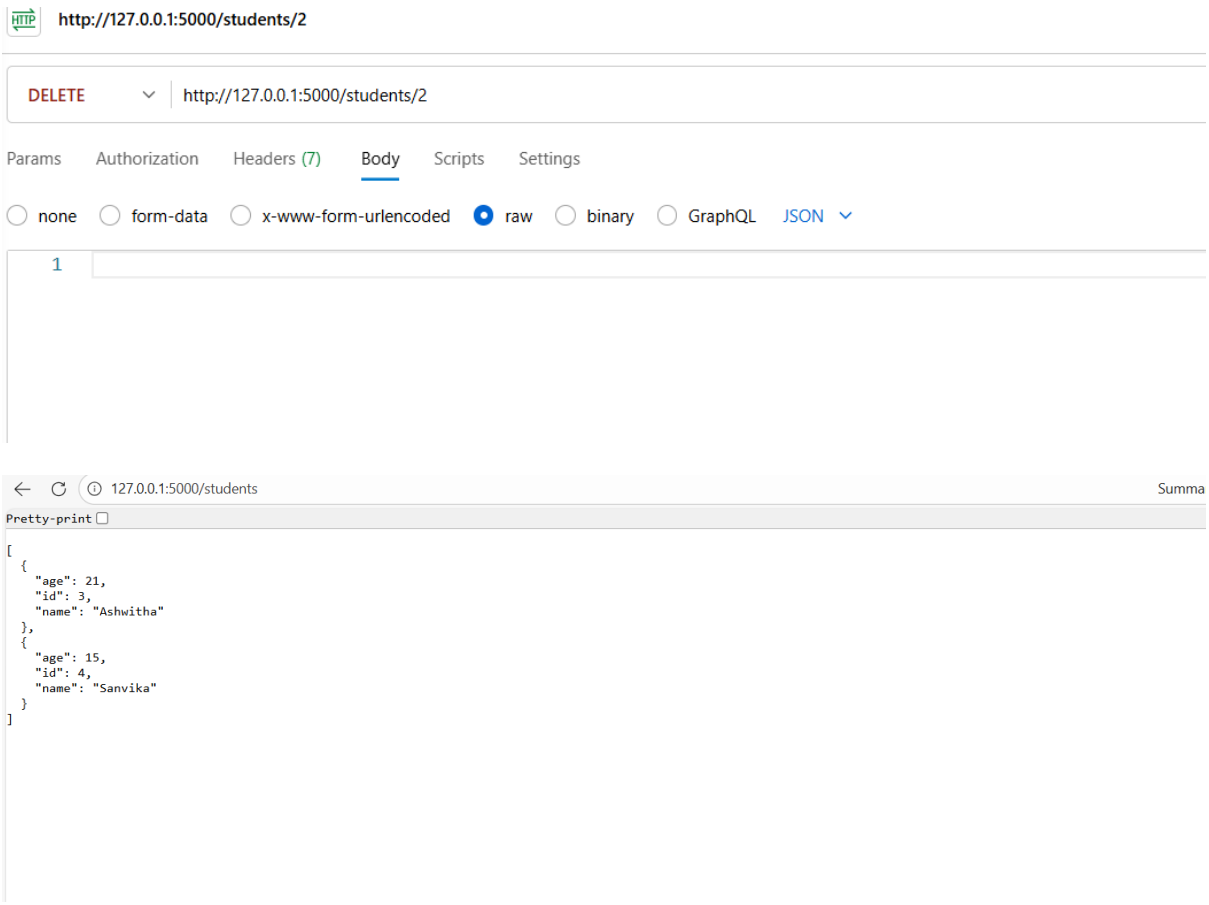
GET


http://127.0.0.1:5000/students
Save
No Environment
GET
http://127.0.0.1:5000/students
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 6 ms
Size: 406 B
Pretty
Raw
Preview
JSON
[
{
"age": 21,
"id": 1,
"name": "Pavani"
},
{
"age": 20,
"id": 2,
"name": "Manisha"
},
{
"age": 21,
"id": 3,
"name": "Ashwitha"
},
{
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS

PUT


http://127.0.0.1:5000/students/1
Save
No Environment
PUT
http://127.0.0.1:5000/students/1
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"name":"Pallavi"
}
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 8 ms
Size: 190 B
Pretty
Raw
Preview
JSON
{
"name": "Pallavi"
}

DELETE



EXPLANATION:

- Built a RESTful API using Flask to perform CRUD operations on student records.
- Used an in-memory data structure (dictionary/list) to store and manage student data.
- Implemented JSON responses with proper error handling for invalid requests and missing records.

Task 2 – Library Book Management API

Prompt:

develop a python code to develop a RESTful API to handle library booksThe API should support the following operations:

GET /books → Retrieve all books

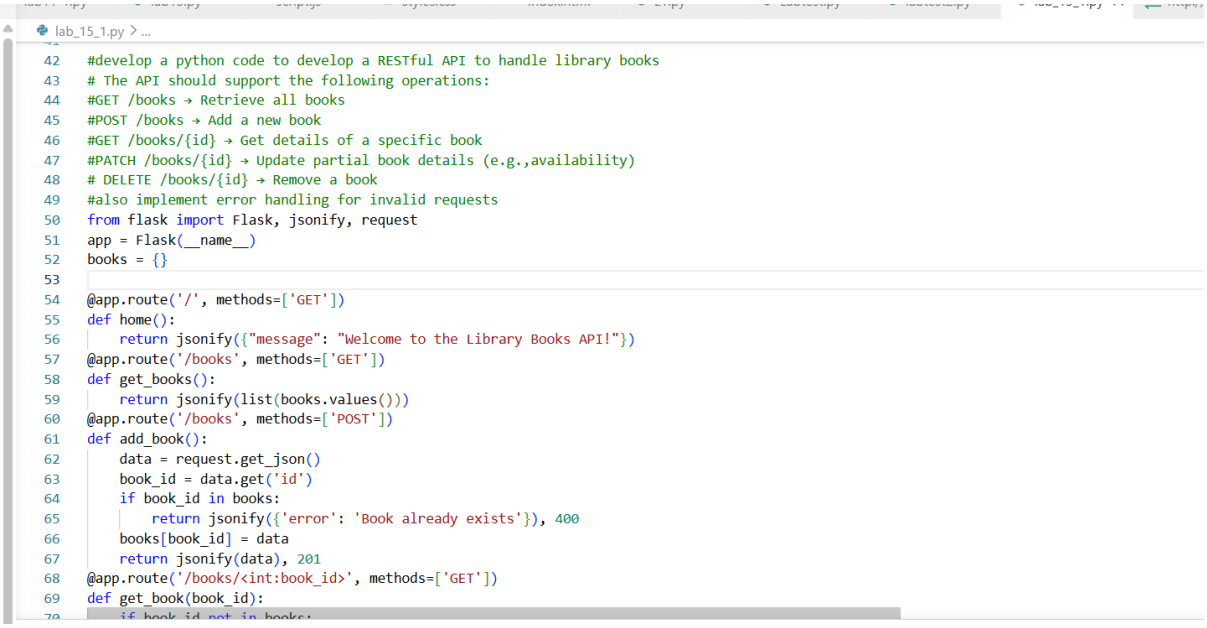
POST /books → Add a new book

GET /books/{id} → Get details of a specific book

PATCH /books/{id} → Update partial book details (e.g.,availability)

DELETE /books/{id} → Remove a book and also implement error handling for invalid requests

CODE:



```python
#develop a python code to develop a RESTful API to handle library books
# The API should support the following operations:
#GET /books → Retrieve all books
#POST /books → Add a new book
#GET /books/{id} → Get details of a specific book
#PATCH /books/{id} → Update partial book details (e.g.,availability)
# DELETE /books/{id} → Remove a book
#also implement error handling for invalid requests
from flask import Flask, jsonify, request
app = Flask(__name__)
books = {}

@app.route('/', methods=['GET'])
def home():
    return jsonify({"message": "Welcome to the Library Books API!"})
@app.route('/books', methods=['GET'])
def get_books():
    return jsonify(list(books.values()))
@app.route('/books', methods=['POST'])
def add_book():
    data = request.get_json()
    book_id = data.get('id')
    if book_id in books:
        return jsonify({'error': 'Book already exists'}), 400
    books[book_id] = data
    return jsonify(data), 201
@app.route('/books/<int:book_id>', methods=['GET'])
def get_book(book_id):
```

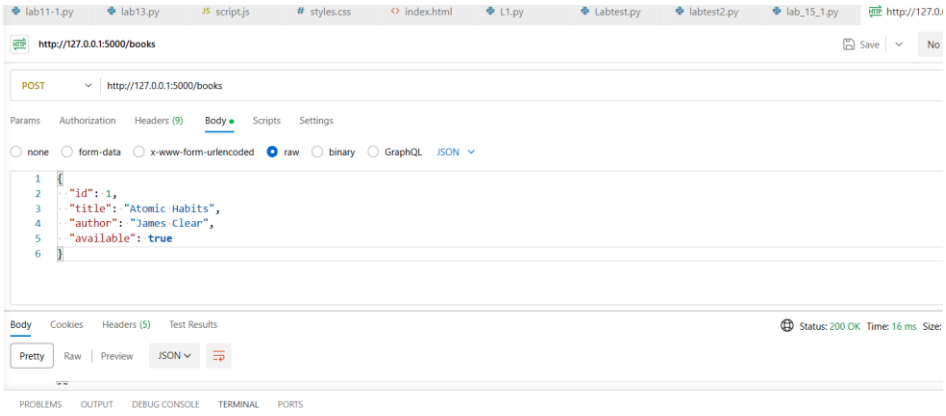
```python
def add_book():
    return jsonify(data), 201
@app.route('/books/<int:book_id>', methods=['GET'])
def get_book(book_id):
    if book_id not in books:
        return jsonify({'error': 'Book not found'}), 404
    return jsonify(books[book_id])
@app.route('/books/<int:book_id>', methods=['PATCH'])
def update_book(book_id):
    if book_id not in books:
        return jsonify({'error': 'Book not found'}), 404
    data = request.get_json()
    books[book_id].update(data)
    return jsonify(books[book_id])
@app.route('/books/<int:book_id>', methods=['DELETE'])
def delete_book(book_id):
    if book_id not in books:
        return jsonify({'error': 'Book not found'}), 404
    del books[book_id]
    return jsonify({'message': 'Book deleted'})
if __name__ == '__main__':
    app.run(debug=True,host='0.0.0.0', port=5000)
```

OUTPUT:

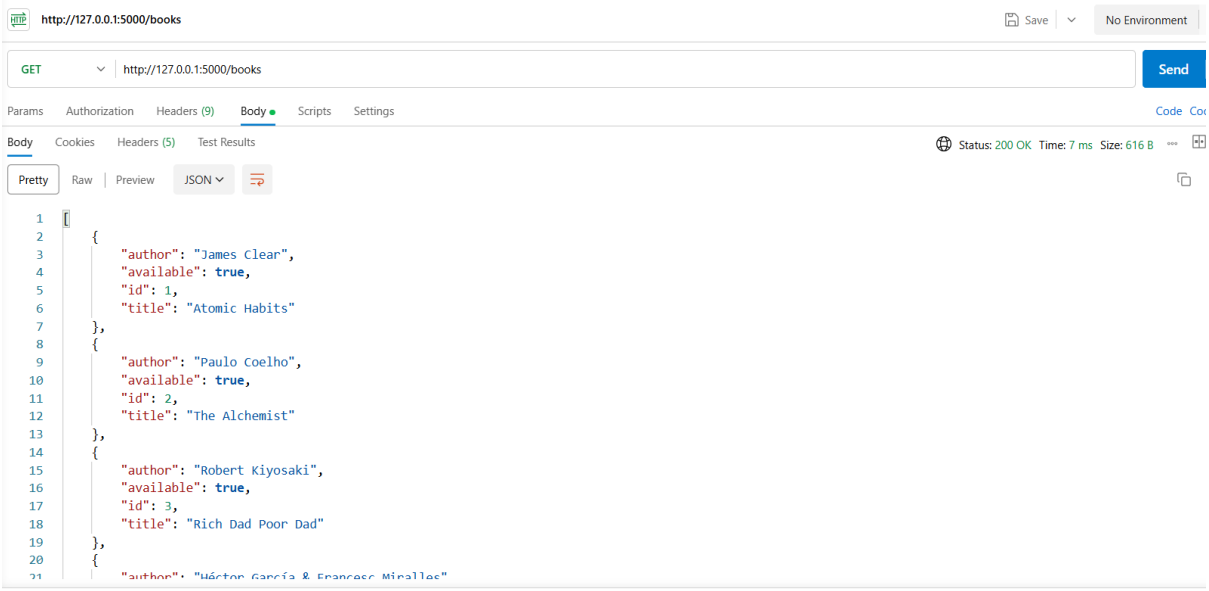
Pretty-print


```
{
  "message": "Welcome to the Library Books API!"
}
```

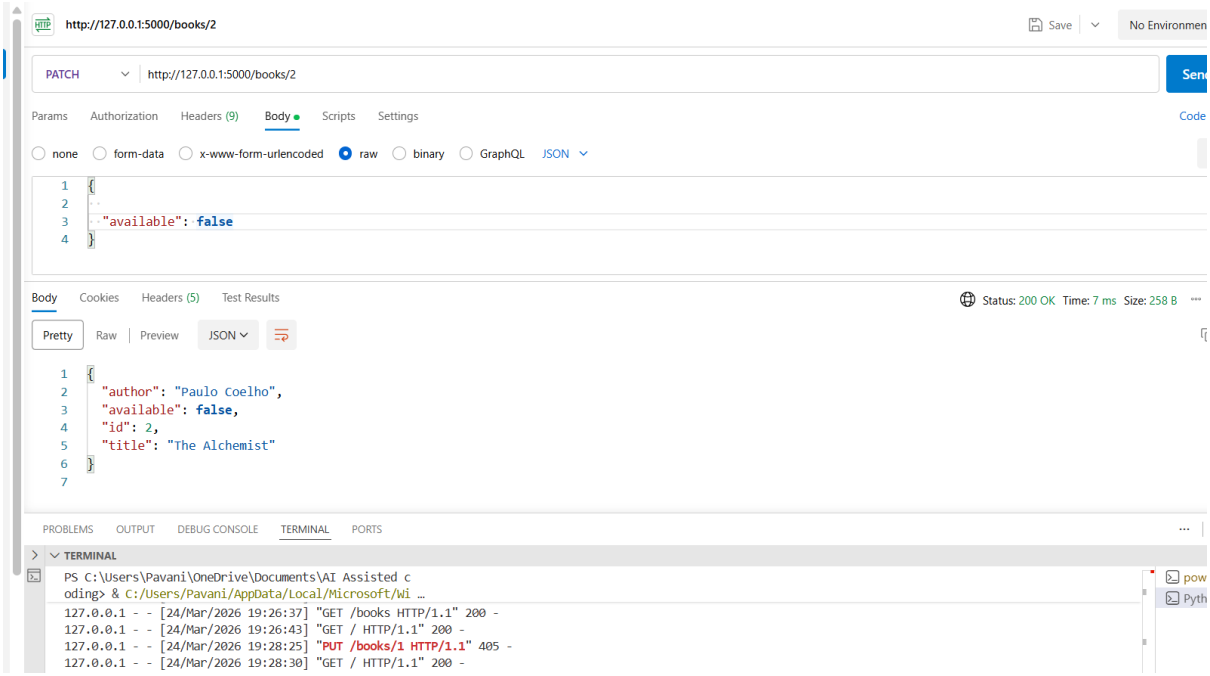

POST:



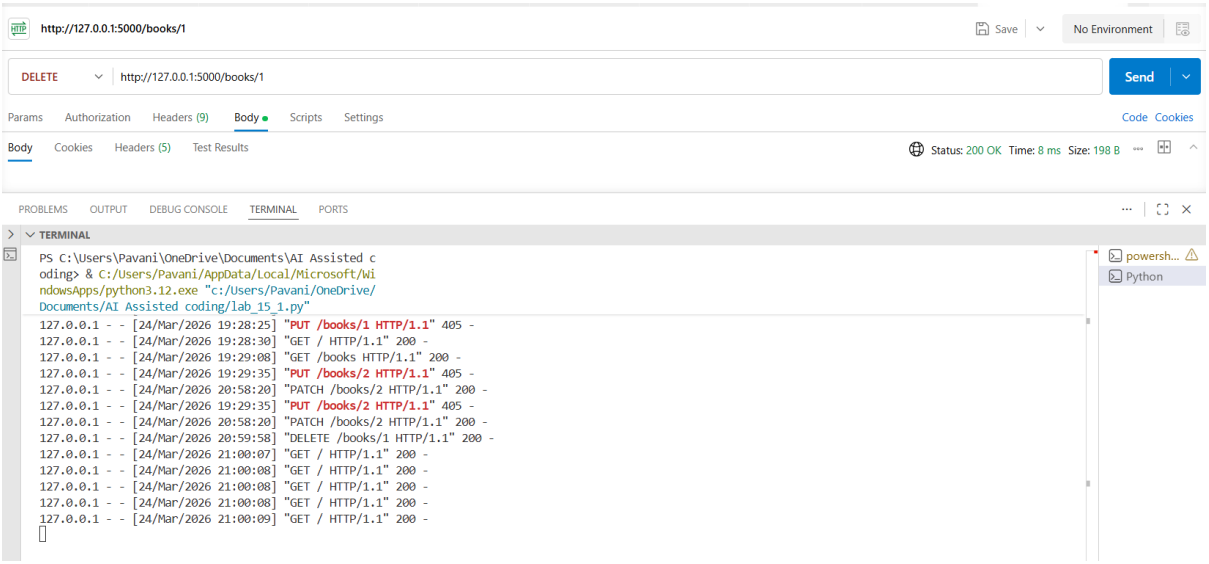
GET

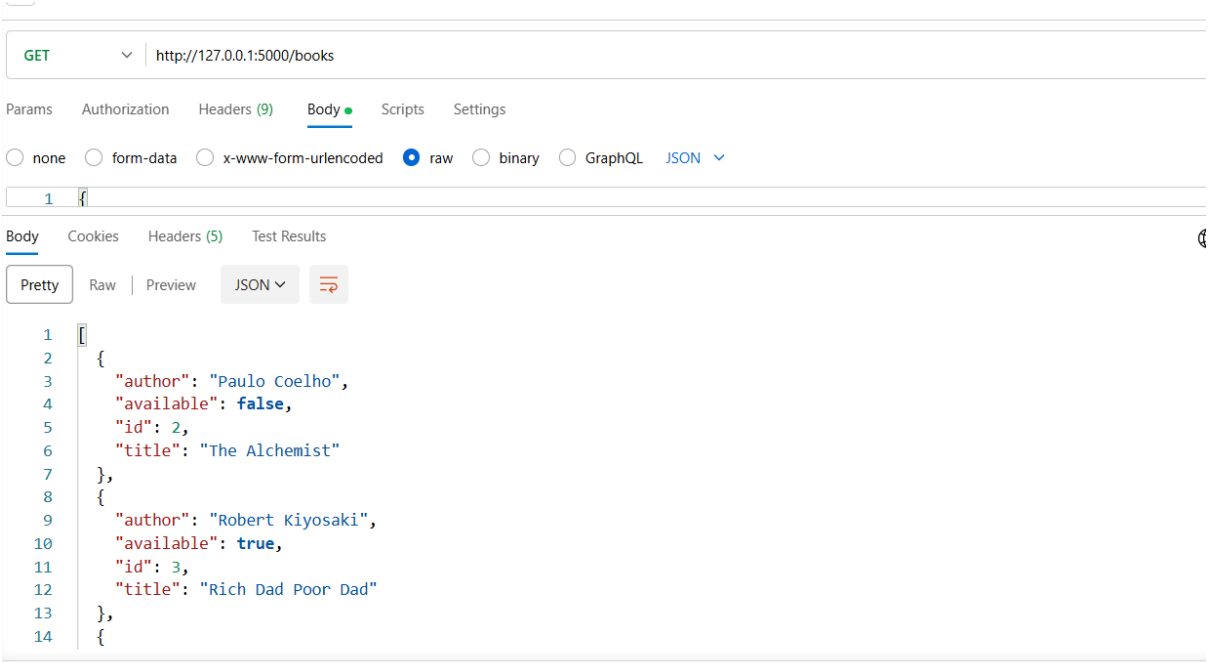


PATCH/PUT



DELETE





EXPLANATION:

- Designed a RESTful API using Flask to perform CRUD operations on library books.
- Implemented in-memory data storage using a Python dictionary for efficient data handling.
- Incorporated error handling and validation to ensure reliable and robust API responses.

Task 3 – Employee Payroll API

PROMPT:

develop a python code to create an API for managing employees and their salaries The API should support the following operations:GET /employees → List all employees POST /employees → Add a new employee with salary details PUT /employees/{id}/salary → Update salary of an employee DELETE /employees/{id} → Remove employee from system

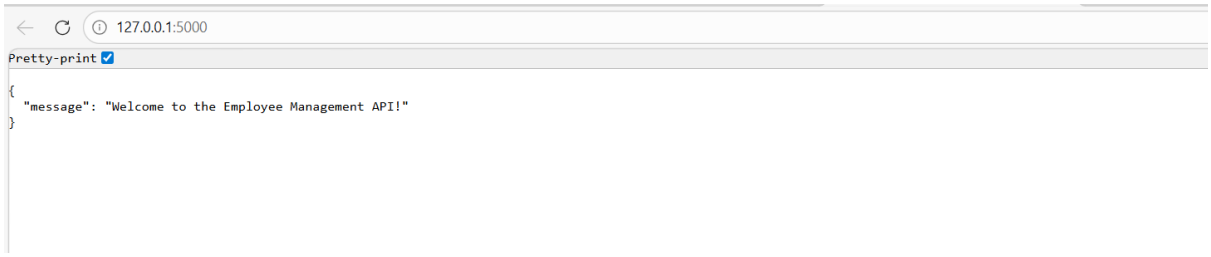
CODE:

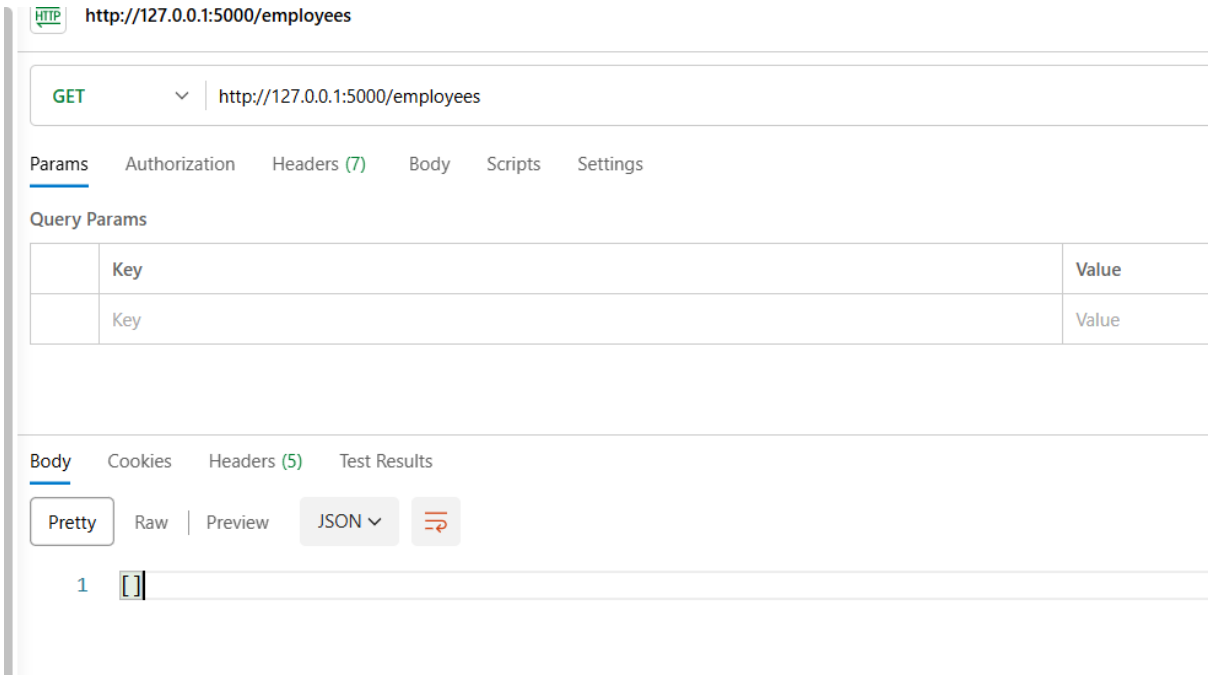
```python

#develop a python code to create an API for managing employees and their salaries
# The API should support the following operations:
#GET /employees → List all employees
#POST /employees → Add a new employee with salary details
#PUT /employees/{id}/salary → Update salary of an employee
#DELETE /employees/{id} → Remove employee from system
from flask import Flask, jsonify, request
app = Flask(__name__)
employees = {}
@app.route('/', methods=['GET'])
def home():
    return jsonify({"message": "Welcome to the Employee Management API!"})
@app.route('/employees', methods=['GET'])
def get_employees():
    return jsonify(list(employees.values()))
@app.route('/employees', methods=['POST'])
def add_employee():
    data = request.get_json()
    employee_id = data.get('id')
    if employee_id in employees:
        return jsonify({'error': 'Employee already exists'}), 400
    employees[employee_id] = data
    return jsonify(data), 201
@app.route('/employees/<int:employee_id>/salary', methods=['PUT'])
def update_salary(employee_id):
    if employee_id not in employees:
        return jsonify({'error': 'Employee not found'}), 404
    data = request.get_json()
    employees[employee_id]['salary'] = data.get('salary')
    return jsonify(employees[employee_id])
@app.route('/employees/<int:employee_id>', methods=['DELETE'])
def delete_employee(employee_id):
    if employee_id not in employees:
        return jsonify({'error': 'Employee not found'}), 404
    del employees[employee_id]
```

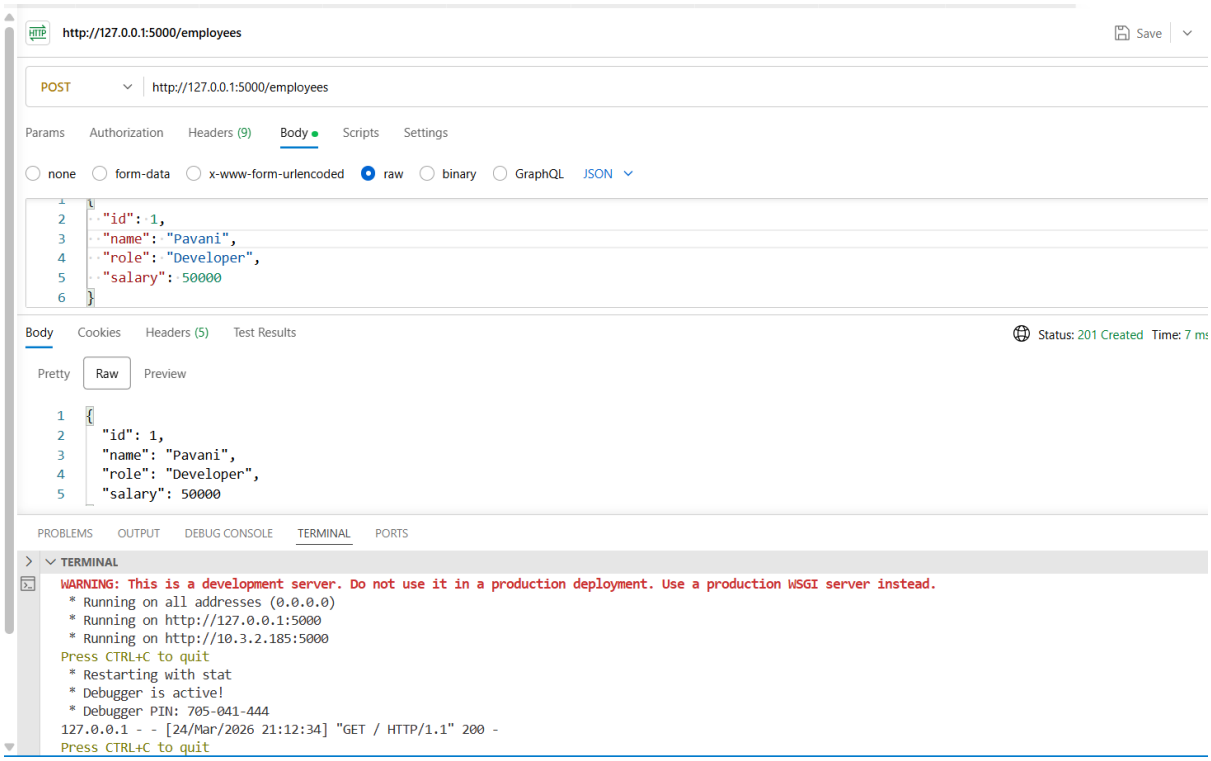
```python
    employee_id = data.get('id')
    if employee_id in employees:
        return jsonify({'error': 'Employee already exists'}), 400
    employees[employee_id] = data
    return jsonify(data), 201
@app.route('/employees/<int:employee_id>/salary', methods=['PUT'])
def update_salary(employee_id):
    if employee_id not in employees:
        return jsonify({'error': 'Employee not found'}), 404
    data = request.get_json()
    employees[employee_id]['salary'] = data.get('salary')
    return jsonify(employees[employee_id])
@app.route('/employees/<int:employee_id>', methods=['DELETE'])
def delete_employee(employee_id):
    if employee_id not in employees:
        return jsonify({'error': 'Employee not found'}), 404
    del employees[employee_id]
    return jsonify({'message': 'Employee deleted'})
if __name__ == '__main__':
    app.run(debug=True,host='0.0.0.0', port=5000)

```

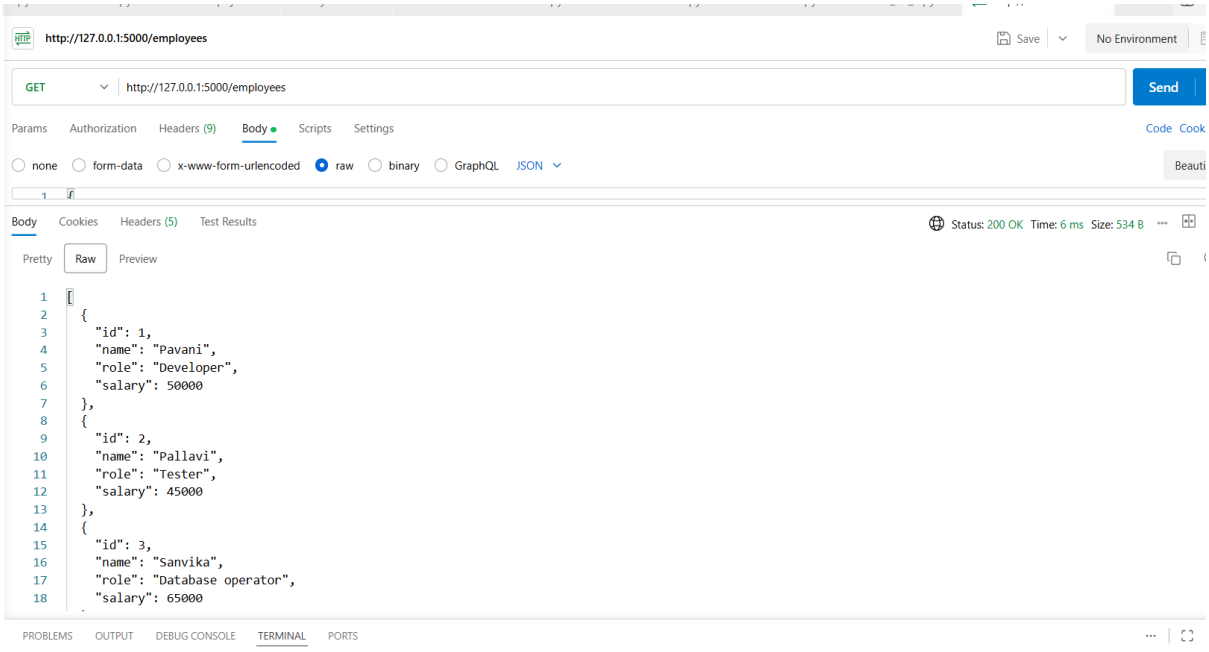
```
OBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

TERMINAL
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.3.2.185:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 705-041-444
127.0.0.1 - - [24/Mar/2026 21:12:34] "GET / HTTP/1.1" 200 -
Press CTRL+C to quit
```

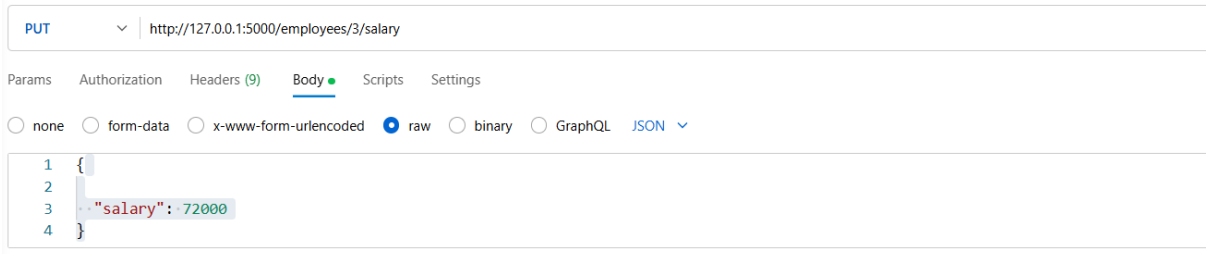
OUTPUT:


127.0.0.1:5000
Pretty-print
{
"message": "Welcome to the Employee Management API!"
}

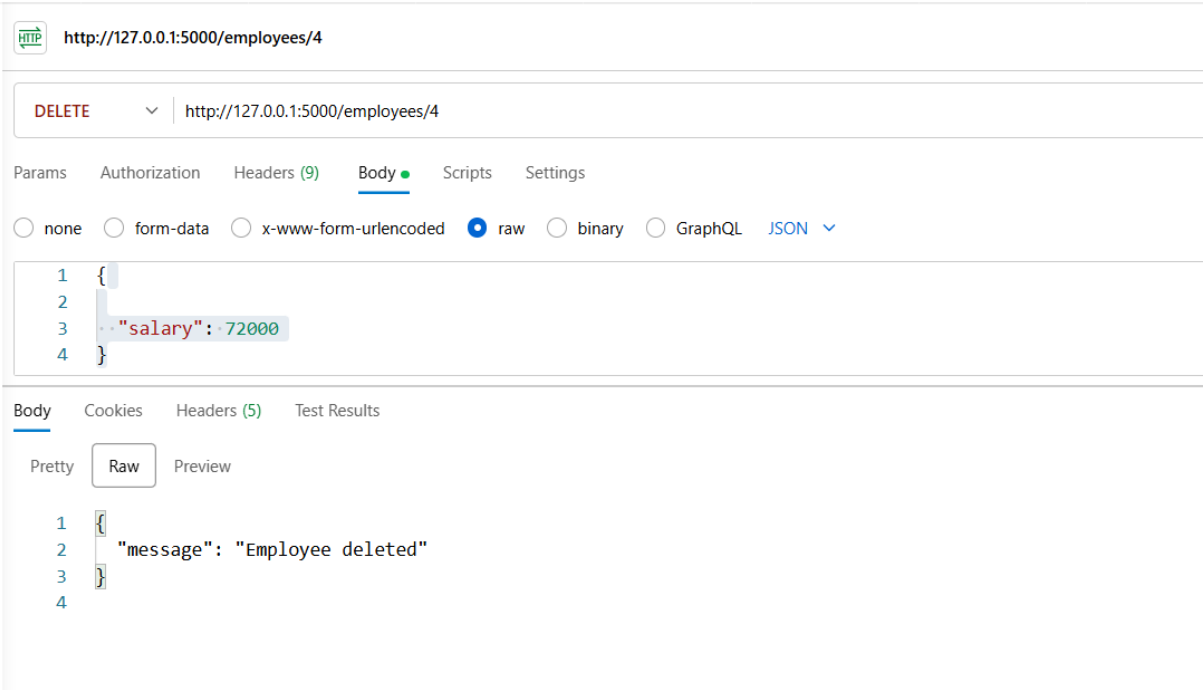

http://127.0.0.1:5000/employees
GET
http://127.0.0.1:5000/employees
Params
Authorization
Headers (7)
Body
Scripts
Settings
Query Params
Key
Value
Key
Value
Body
Cookies
Headers (5)
Test Results
Pretty
Raw
Preview
JSON
1
[]


http://127.0.0.1:5000/employees
Save
POST
http://127.0.0.1:5000/employees
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"id": 1,
"name": "Pavani",
"role": "Developer",
"salary": 50000
}
Body
Cookies
Headers (5)
Test Results
Status: 201 Created
Time: 7 ms
Pretty
Raw
Preview
{
"id": 1,
"name": "Pavani",
"role": "Developer",
"salary": 50000
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
TERMINAL
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.3.2.185:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 705-041-444
127.0.0.1 - - [24/Mar/2026 21:12:34] "GET / HTTP/1.1" 200 -
Press CTRL+C to quit


http://127.0.0.1:5000/employees
Save
No Environment
GET
http://127.0.0.1:5000/employees
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cook
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beauti
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 6 ms
Size: 534 B
Pretty
Raw
Preview
[
{
"id": 1,
"name": "Pavani",
"role": "Developer",
"salary": 50000
},
{
"id": 2,
"name": "Pallavi",
"role": "Tester",
"salary": 45000
},
{
"id": 3,
"name": "Sanvika",
"role": "Database operator",
"salary": 65000
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS


PUT
http://127.0.0.1:5000/employees/3/salary
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"salary": 72000
}


Body
Cookies
Headers (5)
Test Results
Status
Pretty
Raw
Preview
{
"id": 3,
"name": "Sanvika",
"role": "Database operator",
"salary": 72000
}



EXPLANATION:

- Developed a RESTful API using Flask to manage employee records and their salaries.
- Implemented CRUD operations like adding, viewing, updating salary, and deleting employees.
- Used a Python dictionary as in-memory storage to handle employee data.
- Included JSON responses and basic error handling for invalid requests and missing employees.

Task 4 – Real-Time Application: Online Food Ordering API

PROMPT:

develop a python code to design a simpleAPI for an online food ordering system

The API should support the following operations:

GET /menu → List available dishes

POST /order → Place a new order

GET /order/{id} → Track order status

PUT /order/{id} → Update an existing order (e.g.,change items)

DELETE /order/{id} → Cancel an order

also implement error handling for invalid requests

CODE:

lab_15_1.py > ...

```python
#develop a python code to design a simpleAPI for an online food ordering system
# The API should support the following operations:
#GET /menu → List available dishes
#POST /order → Place a new order
#GET /order/{id} → Track order status
#PUT /order/{id} → Update an existing order (e.g.,change items)
#DELETE /order/{id} → Cancel an order
#also implement error handling for invalid requests
from flask import Flask, jsonify, request
app = Flask(__name__)
menu = {
    1: {"name": "Pizza", "price": 10.99},
    2: {"name": "Burger", "price": 7.99},
    3: {"name": "Pasta", "price": 8.99}
}
orders = {}
@app.route('/', methods=['GET'])
def home():
    return jsonify({"message": "Welcome to the Online Food Ordering API!"})
@app.route('/menu', methods=['GET'])
def get_menu():
    return jsonify(menu)
@app.route('/order', methods=['POST'])
def place_order():
    data = request.get_json()
    order_id = len(orders) + 1
    orders[order_id] = {
        "items": data.get('items'),
        "status": "placed"
    }
    return jsonify({"order_id": order_id, "status": "placed"}), 201
```

lab_15_1.py > ...

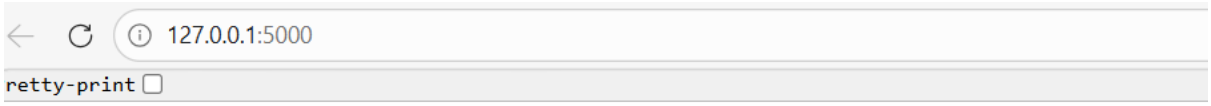
```python
    return jsonify({"order_id": order_id, "status": "placed"}), 201
@app.route('/order/<int:order_id>', methods=['GET'])
def track_order(order_id):
    if order_id not in orders:
        return jsonify({'error': 'Order not found'}), 404
    return jsonify(orders[order_id])
@app.route('/order/<int:order_id>', methods=['PUT'])
def update_order(order_id):
    if order_id not in orders:
        return jsonify({'error': 'Order not found'}), 404
    data = request.get_json()
    orders[order_id]['items'] = data.get('items', orders[order_id]['items'])
    return jsonify(orders[order_id])
@app.route('/order/<int:order_id>', methods=['DELETE'])
def cancel_order(order_id):
    if order_id not in orders:
        return jsonify({'error': 'Order not found'}), 404
    del orders[order_id]
    return jsonify({'message': 'Order cancelled'})
if __name__ == '__main__':
    app.run(debug=True,host='0.0.0.0', port=5000)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

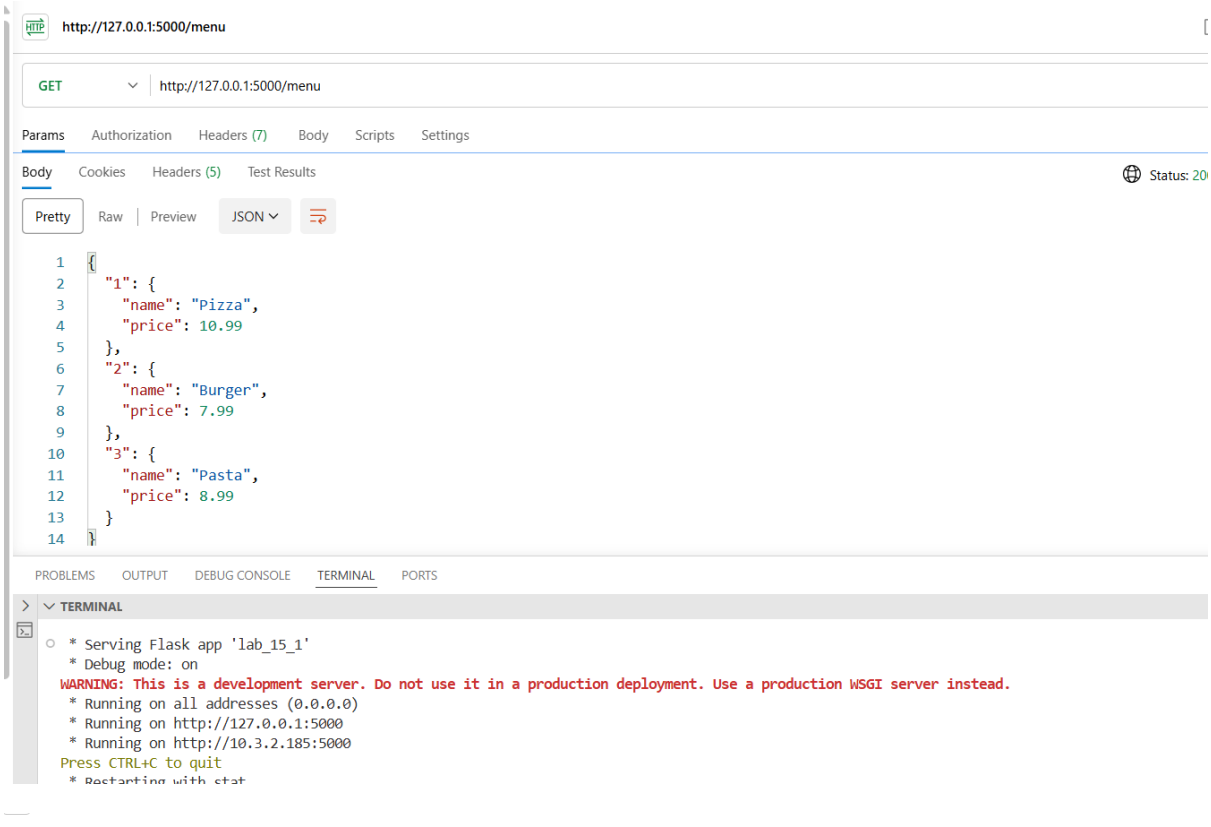
TERMINAL

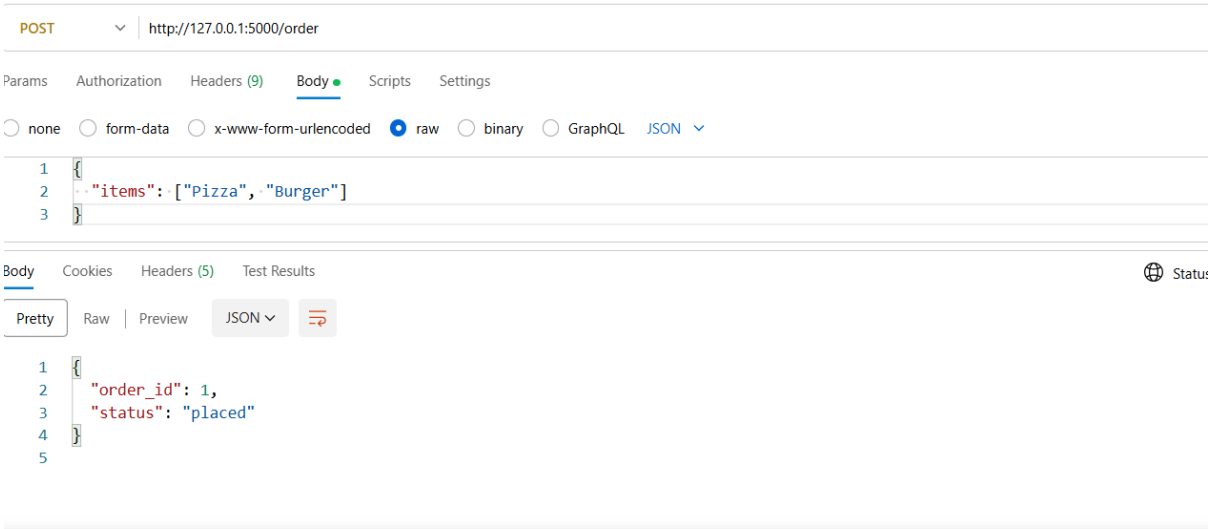
```
 * Serving Flask app 'lab_15_1'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.3.2.185:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 705-041-444
127.0.0.1 - - [24/Mar/2026 21:28:53] "GET / HTTP/1.1" 200 -
```

OUTPUT:




```
"message": "Welcome to the Online Food Ordering API!"
```



PUT http://127.0.0.1:5000/order/1

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "items": ["Pasta", "Burger"]
}
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
{
  "order_id": 1,
  "status": "placed"
}
```

http://127.0.0.1:5000/order/1

DELETE http://127.0.0.1:5000/order/1

Params Authorization Headers (7) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not h

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
{
    "message": "Order cancelled"
}
```

EXPLANATION:

- Developed a RESTful API using Flask to manage menu items and customer orders in an online food ordering system.
- Implemented CRUD operations: viewing menu (GET), placing orders (POST), updating orders (PUT), and cancelling orders (DELETE).
- Used in-memory data structures (dictionaries) to store menu and order details.
- Ensured JSON responses and added basic error handling for invalid requests and missing orders.